

INFOSYS SPRINGBOARD

INTERNSHIP PROJECT REPORT

AI-DRIVEN ENTERPRISE PROJECT INTELLIGENCE & RISK MANAGEMENT PLATFORM

An intelligent platform for document-grounded project analysis,
risk identification, project health assessment, and project intelligence

Abstract

Project information is commonly distributed across proposals, requirements documents, meeting notes, task lists, progress updates, and other development artefacts. Although these documents contain valuable information about project objectives, deliverables, risks, dependencies, and progress, manually consolidating the information is difficult and does not scale well as the project evolves. The AI Project Intelligence & Risk Advisor addresses this problem by transforming project documents into a unified, searchable knowledge layer and applying specialised analytical workflows to derive project intelligence.

The platform uses a full-stack architecture consisting of a React/Vite frontend and a Python/FastAPI backend. Uploaded project documents are stored through the application's storage layer and processed using a document loader based on Docling. The resulting content is represented as Markdown, segmented using recursive character-based chunking, enriched with project and document metadata, embedded using the configured embedding service, and stored in a project-specific Qdrant collection. Structured application and analysis state is maintained in MongoDB through Beanie models.

A multi-stage analysis workflow coordinates dedicated agents for scope extraction, risk analysis, project health calculation, document auditing, document generation, and conversational question answering. Scope analysis uses multiple retrieval queries to collect evidence about objectives, deliverables, timeline, stakeholders, and exclusions. Risk analysis performs broader retrieval focused on challenges, unclear requirements, deadlines, resources, dependencies, technical complexity, and integration concerns. The health calculation is deterministic and combines scope clarity, documentation completeness, risk density, and schedule-risk measures.

The document generation component supports executive summaries, user stories, risk registers, and sprint plans. It audits existing generated documents and generates only the missing types. The conversational assistant performs semantic retrieval within the selected project, combines retrieved sources with recent conversation history, and passes the grounded context to the configured language model. The current frontend provides project creation and listing, document management, analysis, generated-data, report, chat, overview, settings, document viewing, PDF download, and theme-related interface components.

The resulting system provides an integrated project intelligence workflow rather than a simple document search tool. It connects source documents, structured analysis, generated artefacts, and conversational access while maintaining project-specific retrieval boundaries. The platform is designed to support informed project decisions by making important project information easier to discover, interpret, and review.

Keywords: Project Intelligence, Risk Analysis, Retrieval-Augmented Generation, Large Language Models, Semantic Search, Multi-Agent Workflow, Project Health, Document Intelligence, FastAPI, React, Qdrant, MongoDB.

Table of Contents

- 01. Introduction
- 02. Existing Approach and Proposed Solution
- 03. Requirements Analysis
- 04. Technology and Development Environment
- 05. System Architecture
- 06. Document Ingestion and Knowledge Base Construction
- 07. Retrieval-Augmented Generation Pipeline
- 08. Project Scope Extraction
- 09. Risk Detection and Risk Analysis
- 10. Project Health Scoring
- 11. Automated Project Document Generation
- 12. Conversational Project Intelligence Assistant
- 13. Project Workspace and Dashboard
- 14. Data Model and Persistence
- 15. API and Backend Implementation
- 16. Authentication and Security
- 17. End-to-End System Workflow
- 18. Testing and Evaluation Strategy
- 19. Results and Discussion
- 20. Limitations and Future Scope
- 21. Conclusion
- 22. References
- 23. Appendix A – API Summary
- 24. Appendix B – Configuration and Deployment
- 25. Appendix C – Data Structures and Output Formats
- 26. Appendix D – Glossary

1. Introduction

1.1 Background

Software development generates a substantial amount of project information throughout the development lifecycle. Initial project proposals describe the problem, objectives, and intended outcomes; requirements documents define expected functionality; meeting records capture decisions, dependencies, and unresolved concerns; task lists describe planned and completed work; and progress reports communicate the current state of development. Although these artefacts contain valuable project knowledge, the information is often distributed across different documents and formats, making it difficult to obtain a unified view of the project.

Traditional project document repositories primarily provide mechanisms for storing and retrieving files. They generally do not transform a collection of project artefacts into structured intelligence about project scope, risks, documentation completeness, or overall health. Consequently, team members may need to manually inspect multiple documents to determine what the project is expected to deliver, which risks are currently visible, what information is missing, or whether schedule-related concerns exist.

Recent developments in artificial intelligence, semantic retrieval, and large language models provide an opportunity to address this problem. Retrieval-Augmented Generation (RAG) enables relevant information to be retrieved from project-specific sources before generating an answer, reducing dependence on general model knowledge. This approach is particularly suitable for project intelligence because project information is specific to each workspace, evolves throughout development, and needs to remain grounded in the actual project documentation.

The AI Project Intelligence & Risk Advisor applies these concepts to create a unified intelligence layer over project documentation. The platform combines document processing, semantic retrieval, structured AI analysis, deterministic computation, persistent storage, and a web-based interface to transform scattered project artefacts into accessible project intelligence.

1.2 Problem Statement

Software development teams and project groups frequently encounter incomplete planning, missed deadlines, undetected risks, and insufficient documentation. Although teams continuously create project-related artefacts such as project proposals, requirements documents, meeting notes, sprint updates, task lists, and progress reports, the information contained in these artefacts is often scattered across files and formats. Extracting actionable information from this collection manually is time-consuming and may be overlooked during day-to-day development.

The lack of a unified view can result in important project signals remaining disconnected. For example, an unresolved dependency may be recorded in a meeting note, incomplete requirements may appear in a progress report, and schedule-related concerns may be present in a task list. When these pieces of information are considered independently, the overall project situation may not be immediately visible. This can lead to reactive problem-solving rather than proactive project management.

The objective of the AI Project Intelligence & Risk Advisor is to provide an intelligent platform in which existing project documentation can be transformed into a unified, searchable project knowledge layer. The system accepts project artefacts, processes their contents, constructs a project-specific retrieval-oriented knowledge base, and applies specialised AI analysis to extract project scope and deliverables, identify project risks, assess project health, and generate useful project documentation.

The platform further provides a conversational project assistant through which users can ask questions about project status, scope, risks, deliverables, and recommendations. Responses are generated using retrieved project information so that project-specific answers remain connected to the available source material.

The implemented system therefore connects unstructured project documentation with structured project intelligence through document processing, semantic retrieval, large language model reasoning, structured persistence, deterministic health computation, background analysis, authentication, and an interactive web interface.

1.3 Motivation

The primary motivation of the project is to enable proactive awareness of project status and potential delivery concerns.

Project information is normally produced continuously rather than in a single structured source. A proposal may describe the intended scope, while later meeting notes, progress reports, and task records contain information about changes, dependencies, incomplete work, or emerging concerns. Manually connecting these signals requires repeated reading and interpretation of project documents.

An intelligent project analysis platform can reduce this effort by bringing relevant information together before analysis. By combining semantic retrieval with structured AI processing, the system can identify information relevant to project scope and risk assessment and present it in a form that is easier for project teams to review.

The motivation is therefore not to replace project management activities, but to provide an intelligence layer that helps teams understand available project evidence more efficiently and identify potential concerns earlier.

1.4 Objectives

The major objectives of the project are:

1. To develop a document ingestion workflow capable of accepting common project artefacts and converting their contents into a form suitable for downstream analysis.
2. To construct a project-specific Retrieval-Augmented Generation knowledge base using document chunking, embeddings, and vector search.
3. To automatically extract project scope information including objectives, deliverables, timeline, stakeholders, and out-of-scope information from available project documentation.
4. To identify project risks from project evidence and classify them according to category, severity, probability, and impact while providing associated mitigation information and source context.
5. To compute an interpretable project health score using structured project and risk information through deterministic calculations.
6. To generate useful project artefacts, including executive summaries, user stories, risk registers, and sprint plans, based on available project information.
7. To provide a conversational project intelligence assistant capable of answering project-related questions using retrieved project sources and indicating when sufficient information is unavailable.
8. To provide a web-based project workspace for project creation, document management, analysis execution, report viewing, generated documents, and conversational interaction.

9. To support authenticated access to protected project operations while maintaining project-level separation of project documents and analytical information.

1.5 Scope

The scope of the implemented system covers the complete document-to-analysis workflow available in the application.

The platform supports project workspace management and authenticated access to project operations. Users can create and manage projects and upload project documentation for processing. Uploaded documents are stored and processed before their contents are divided into suitable chunks, converted into vector representations, and stored in a project-specific vector collection.

The analytical workflow includes project scope extraction, risk identification and classification, deterministic project health scoring, and automated generation of project artefacts. The system also maintains analysis status and persists the resulting analytical report and associated generated information.

The generated project artefacts currently supported by the implementation are:

- Executive Summary
- User Stories
- Risk Register
- Sprint Plan

The conversational assistant provides project-level question answering using semantic retrieval over uploaded documents and generated analytical artefacts. The web interface provides access to project overview information, documents, analysis results, generated data, reports, and the conversational assistant.

The scope is limited to the capabilities represented in the current implementation. The system does not claim a separate standalone blocker/action-item analysis agent or a separately trained time-series forecasting model as part of the implemented functionality.

1.6 Key Expected Outcomes

The project is expected to provide the following outcomes:

- A unified project knowledge base derived from uploaded project documentation.
- Structured project scope information extracted from available project sources.
- Evidence-based risk entries containing category, severity, probability, impact, mitigation, and source context.
- A reproducible project health score accompanied by a component-wise breakdown.
- Automatically generated project artefacts stored as part of the project analysis.
- A document-grounded conversational assistant capable of retrieving relevant project sources before generating responses.

- A web-based project workspace that brings project documents and analytical results together within a single interface.
- Persistent project-level analytical information that can be retrieved without unnecessarily repeating the complete analysis workflow.

1.7 Design Principles

The implementation follows several design principles that guide the architecture and analytical workflow:

- **Evidence grounding:** Project-specific analytical responses should be based on retrieved project evidence rather than unsupported assumptions.
- **Project isolation:** Documents and vector representations are maintained at the project level so that information from one project does not become context for another project.
- **Structured outputs:** Important analytical results are represented using structured data models rather than relying only on unstructured language model responses.
- **Deterministic computation:** Calculations such as project health scoring are performed deterministically rather than being delegated entirely to a language model.
- **Persistent analysis:** Analytical results are stored so that they can be retrieved and presented without unnecessarily repeating model operations.
- **Modular analysis:** Scope extraction, risk analysis, health computation, document generation, and conversational assistance are implemented as separate responsibilities.
- **Asynchronous processing:** Long-running analysis operations are separated from the initiating user request so that the application can track processing status while the analysis is performed.
- **Traceability:** Analytical risk information retains source context so that users can relate identified risks to the underlying project evidence.

2. Existing Approach and Proposed Solution

2.1 Existing Approach

A conventional project documentation workflow stores individual documents in folders or shared repositories. Team members search for files manually, read relevant sections, and consolidate findings mentally or in separate notes. Risk registers may be maintained independently from requirements and progress records, and generated project documents may require repeated manual preparation.

Keyword search is useful when the exact terminology is known, but it is less effective when the same concept is expressed differently across documents. For example, schedule pressure may appear as an overdue task, a delayed milestone, a dependency awaiting another team, or a statement that a release date is at risk.

2.2 Proposed Solution

The proposed solution introduces a project-level intelligence layer between raw documentation and project users. Uploaded artefacts are processed into searchable passages. Semantic retrieval is then used by specialised agents to collect evidence relevant to each analytical task. The agents produce structured outputs that are stored in the analysis report and made available to the interface and conversational assistant.

2.3 Why RAG is Used

The project does not need to retrain a language model whenever project documentation changes. Instead, relevant information is retrieved at query time. This allows the knowledge base to evolve as new documents are uploaded and permits answers to remain tied to the current project corpus.

2.4 Proposed Processing Stages

1. Create or select a project workspace.
2. Upload project documentation.
3. Store document metadata and the document object through the storage layer.
4. Process the document into a common text representation.
5. Split the processed content into overlapping chunks.
6. Attach project and document metadata to each chunk.
7. Generate embeddings and insert the chunks into the project's Qdrant collection.
8. Run the analytical pipeline when the project is ready.
9. Extract project scope and identify risks.
10. Calculate project health from structured analytical outputs.
11. Audit existing generated documents and generate missing document types.
12. Persist the completed analysis report and synchronise analytical artefacts into the vector index.
13. Allow the user to inspect the results or ask grounded project questions.

3. Requirements Analysis

3.1 Functional Requirements

ID	Area	Requirement
FR-01	Authentication	Provide authenticated access to protected application operations.
FR-02	Project Management	Create, list, inspect, and manage project workspaces.

FR-03	Document Management	Upload, list, inspect, and manage project documents.
FR-04	Document Processing	Process uploaded files into a common representation.
FR-05	Semantic Indexing	Chunk, embed, and store project content in Qdrant.
FR-06	Scope Analysis	Extract project name, objectives, deliverables, timeline, stakeholders, exclusions, and summary.
FR-07	Risk Analysis	Identify risks and provide category, severity, probability, impact, mitigation, and source context.
FR-08	Health Assessment	Calculate a project health score and component breakdown.
FR-09	Document Audit	Determine which supported generated document types already exist and which are missing.
FR-10	Document Generation	Generate missing executive summaries, user stories, risk registers, and sprint plans.
FR-11	Analysis Report	Persist and retrieve the completed analysis report.
FR-12	Analysis Status	Expose pipeline status and current processing step.
FR-13	Conversational Assistant	Answer project-specific questions using semantic retrieval.
FR-14	Source Awareness	Return source labels for retrieved conversational context.
FR-15	Generated Data	Display generated documents through the project workspace.
FR-16	Document Viewing	Provide document viewing capabilities through the frontend.
FR-17	PDF Output	Provide frontend support for PDF download/export of displayed content.

3.2 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Security	Credentials and protected operations must be handled through an authentication mechanism.
NFR-02	Isolation	Semantic retrieval must be scoped to the selected project.
NFR-03	Groundedness	Project-specific AI responses should use retrieved project context.
NFR-04	Reliability	Document failures should be recorded against the affected document.
NFR-05	Performance	Long-running analysis should be executed without keeping the initiating API request open.
NFR-06	Maintainability	AI agents, RAG components, routes, models, and storage should remain modular.
NFR-07	Reproducibility	The health score should be deterministic for the same structured inputs.
NFR-08	Usability	The interface should expose project documents, analysis, reports, generated documents, and chat in a coherent workspace.

NFR-09	Configurability	Database, AI, storage, upload, backend, and authentication settings should be externally configurable.
NFR-10	Scalability	Structured persistence and vector retrieval should be independently manageable.

3.3 Constraints

- The quality of generated analysis depends on the quality and completeness of the uploaded project documentation.
- Semantic retrieval and generative analysis require external AI/vector services configured for the application.
- Generated project artefacts should be reviewed by a human before being treated as authoritative project documentation.
- AI outputs can contain uncertainty even when the system is instructed to use only supplied context.

4. Technology and Development Environment

Technology	Use in the Project
Python	Backend and AI-processing implementation
FastAPI	Backend REST API and application server
Pydantic	Typed validation and structured analytical data
Beanie	Object-document mapping and MongoDB persistence
MongoDB	Project, document, analysis, chat and user-related state
Docling	Document conversion/processing through the LangChain integration
LangChain	Document, retrieval, embedding and LLM integration components
RecursiveCharacterTextSplitter	Overlapping text chunking
NVIDIA Embeddings	Current embedding implementation
Qdrant	Project-specific vector storage and semantic retrieval
Google Gemini / configured Google Generative AI	Language-model generation for analytical agents and chat
LangGraph	Stateful orchestration of the analytical workflow
React	Frontend application
Vite	Frontend development/build tooling
TypeScript	Frontend implementation
React Markdown	Markdown rendering
PapaParse	CSV parsing support in the frontend dependency set
html2pdf.js	PDF generation/download support in the frontend
JWT	Token-based authentication
S3/MinIO-compatible storage	Document object storage layer
Docker Compose	Multi-service development/deployment environment

4.1 Backend Structure

The current backend is organised under an application package containing agents, configuration, controllers, core components, dependencies, models, RAG components, routes, schemas, storage, and utilities. The separation is important because the project combines conventional application engineering with AI processing. Document processing should not be tightly coupled to frontend concerns, and analytical agents should not need to manage low-level database operations directly.

4.2 Frontend Structure

The frontend contains API access, reusable components, context, hooks, libraries, pages, types, and utilities. The page structure includes project creation and listing, project detail, and tabbed project views. The current tabs include Overview, Documents, Analysis, Chat, Generated Data, Report, and Project Settings.

4.3 External Services

The implementation is configured to communicate with MongoDB, Qdrant, an S3/MinIO-compatible document storage service, and external AI services. The application configuration is therefore intentionally environment-driven. API keys, JWT secrets and storage credentials should be supplied through environment configuration rather than source code.

5. System Architecture

5.1 Architectural Overview

The system can be viewed as four cooperating layers: presentation, application/API, intelligence and persistence. The presentation layer is responsible for user interaction. The application layer exposes controlled APIs and coordinates operations. The intelligence layer performs document retrieval and AI analysis. The persistence layer stores structured state, source documents, generated artefacts, and vector representations.

Layer	Major Components	Responsibility
Presentation	React, Vite, TypeScript, UI components, project tabs	Project interaction, document management, analysis display, generated data, reports and chat
Application/API	FastAPI, routes, controllers, schemas, dependencies	Request validation, access control, orchestration and application operations
Intelligence	RAG pipeline, scope agent, risk agent, health agent, document generator, chat agent, orchestrator	Retrieval, extraction, risk analysis, scoring, generation and grounded Q&A
Persistence	MongoDB/Beanie, S3/MinIO-compatible storage, Qdrant	Structured records, document objects and semantic vectors

5.2 High-Level Architecture

The following logical architecture describes the relationships implemented by the current application. A user interacts with the React interface. The frontend calls the FastAPI backend. The backend retrieves structured state from MongoDB, retrieves document objects through the storage service, uses the RAG pipeline to index project material in Qdrant, and invokes the configured AI services for generative analysis.

Component	Flow
User	↓
React/Vite Frontend	Project creation → document upload → analysis → report/chat
FastAPI Backend	Authentication → routes/controllers → analysis orchestration
RAG Pipeline	Storage → Docling → chunking → embeddings → Qdrant
Analysis Agents	Scope → Risk → Health → Document Audit/Generation
Persistence	MongoDB + S3/MinIO-compatible storage + Qdrant
Response	Structured report → dashboard → generated documents → grounded chat

5.3 Project Isolation

The Qdrant configuration creates one collection per project using a project-specific collection name. The vector size is configured as 2048 and cosine distance is used. The vector store is therefore bound to a project's collection before

similarity search or insertion is performed. This is a key architectural boundary because project documents should not become interchangeable retrieval context.

5.4 Analytical State

The analysis report stores the status of the analysis, errors, extracted scope, risk list, health score, health breakdown, generated documents, existing and missing document types, current pipeline step, raw outputs for debugging, and timestamps. This structure makes the output of the multi-stage workflow persistent rather than transient.

6. Document Ingestion and Knowledge Base Construction

6.1 Supported Document Processing

The current project uses a Docling-based document loader through the LangChain Docling integration. The loader receives a resolved file path and exports the loaded content as Markdown-oriented LangChain Document objects. The repository documentation identifies support for common project formats including PDF, DOCX, XLSX and CSV. The architecture therefore treats heterogeneous source files as inputs to a common downstream processing path.

6.2 Storage-to-Processing Flow

14. The document controller records the uploaded document and its storage location.
15. The ingestion pipeline obtains the stored object and downloads it to a temporary file.
16. The document loader processes the temporary file using Docling.
17. The resulting document objects are passed to the text processor.
18. The text processor creates overlapping chunks.
19. Project, document, filename and file-type metadata are attached to every chunk.
20. The embedding component converts the chunks into vectors.
21. The vectors are inserted into the Qdrant collection associated with the project.
22. The document record is updated with processing status and chunk count.
23. The temporary file is removed in the final cleanup step, including after failures.

Stage	Implementation Detail
Document record	Stores project association, filename, file type, storage path, processing state and related metadata.
Temporary retrieval	Stored object is downloaded to a temporary local path before parsing.
Parsing	Docling loader exports content in Markdown form.
Chunking	Recursive character splitter; source documents use 1000-character chunks with 400-character overlap.
Metadata	Project ID, document ID, filename and file type are attached to chunks.
Embedding	Current implementation uses NVIDIAEmbeddings with the configured NVIDIA embedding model.
Vector store	Qdrant collection is created per project.
Status	Processing state moves through the document pipeline and failure information is recorded.
Cleanup	Temporary files are removed in a finally block.

6.3 Chunking Strategy

The current TextProcessor uses RecursiveCharacterTextSplitter. The main document chunking function defaults to a chunk size of 1000 characters and an overlap of 400 characters. The splitter prioritises paragraph and line boundaries

before falling back to spaces and individual characters. A separate raw-text function uses 1000-character chunks with 200-character overlap and is used for generated-document indexing.

6.4 Failure Handling

If document loading produces no content or chunking produces no chunks, the ingestion operation raises an error. The document record is then marked as failed and the processing error is stored. The temporary file is cleaned up regardless of whether the operation succeeds or fails. This gives the frontend a persistent processing state rather than silently losing a failed ingestion attempt.

7. Retrieval-Augmented Generation Pipeline

7.1 RAG Concept in the Project

The RAG layer provides the connection between source documents and generative AI. Instead of sending an entire project archive to a language model, the system searches the project vector collection for semantically relevant chunks. The retrieved content is then supplied as context to the appropriate analytical agent.

7.2 Embedding Model

The current embedder implementation instantiates `NVIDIAEmbeddings` using the model configured in the source code. The Qdrant configuration reserves a vector size of 2048 and uses cosine distance. The embedding interface exposes asynchronous document and query embedding functions.

7.3 Project-Specific Vector Collection

A collection name is derived from the project identifier. When a vector store is requested for a project, the application ensures that the corresponding collection exists and binds the `LangChain QdrantVectorStore` to that collection. This provides an explicit project boundary for semantic search.

7.4 Indexing Generated Analysis

The vector layer does not stop at the original uploads. After analysis, the application can synchronise the analysis report summary and generated documents into the project's Qdrant collection. The report summary is stored as a compact vector. Generated documents are chunked before embedding so that individual portions can be retrieved later. Deterministic UUID-based identifiers are used for the analytical summary and generated-document chunks.

7.5 Retrieval for Scope Analysis

The scope agent performs three semantic searches: project objectives/goals/deliverables; project scope/timeline/milestones/stakeholders; and out-of-scope requirements. Each query retrieves up to six results. Duplicate passages are removed and the final context is capped at fifteen unique chunks before being supplied to the language model.

7.6 Retrieval for Risk Analysis

The risk agent performs five semantic searches focused on risks, blockers, challenges and delays; incomplete or unclear requirements; deadlines and milestones; resource constraints and dependencies; and technical debt, complexity and integration issues. Each query retrieves up to five results. Duplicate passages are removed and the final context is capped at eighteen unique chunks.

7.7 Retrieval for Conversational Assistance

The chat agent performs a single semantic search with $k=12$ over the selected project's Qdrant collection. The retrieved passages can include uploaded documents, the analysis report summary and generated documents. Source labels are derived from metadata so that the returned response can identify where the retrieved information came from. The most recent six conversational messages are included as multi-turn context.

8. Project Scope Extraction

8.1 Scope Agent Responsibility

The scope agent is responsible for extracting the high-level definition of the project from its documents. It is implemented as a dedicated AI component and also exposed as a node within the analytical workflow.

Output Field	Meaning
project_name	Project name if supported by the source documents.
objectives	List of project objectives supported by the documents.
deliverables	Expected project deliverables identified from the documents.
timeline	Overall timeline or related schedule information when available.
stakeholders	Stakeholders explicitly supported by project evidence.
out_of_scope	Items that the project documentation explicitly excludes.
summary	Two-to-three sentence summary of the extracted scope.

8.2 Grounding Rules

- Use only information supported by the project documents.
- Use null for unavailable single-value fields.
- Use empty lists for unavailable list fields.
- Keep extracted lists factual and grounded.
- Keep the summary concise and tied to the available evidence.

8.3 Structured Output and Fallback

The agent first requests structured output using the ScopeOutput model. If structured parsing fails, it falls back to a conventional language-model response and attempts to parse the returned text as JSON. If that also fails, a short portion of the raw response is retained as the summary. This provides a controlled degradation path instead of silently discarding the analysis.

9. Risk Detection and Risk Analysis

9.1 Risk Agent Responsibility

The risk agent analyses project documentation to identify both explicit and implied risks. The prompt directs the agent to examine missed deadlines, technical complexity, unclear requirements, resource gaps, scope creep, external dependencies, communication issues and incomplete documentation. The implementation asks for five to fifteen risks and requires each risk to be supported by project context.

Field	Description
Title	Short description of the identified risk.
Description	Detailed explanation of the risk.
Category	Technical, resource, schedule, scope, external, or quality.
Severity	Low, medium, high, or critical.

Probability	Low, medium, or high.
Impact	Expected consequence if the risk materialises.
Mitigation	Specific action suggested to prevent or reduce the risk.
Source Context	Document-based quote or paraphrase that triggered the risk.

9.2 Risk Categories

Category	Typical Meaning in the Model
Technical	Technical complexity, technical debt, integration or implementation difficulty.
Resource	Team capacity, staffing or resource limitations.
Schedule	Deadline, milestone, delay or delivery timing concerns.
Scope	Unclear requirements, scope growth or boundary uncertainty.
External	Dependencies outside the direct control of the project team.
Quality	Issues related to incomplete documentation, quality or correctness.

9.3 Severity Levels

Severity	Interpretation
Low	A lower-impact risk that should remain visible and monitored.
Medium	A meaningful risk requiring attention or mitigation.
High	A significant risk with potential to materially affect delivery.
Critical	A severe risk requiring immediate attention and mitigation.

9.4 Structured Parsing

The risk agent first attempts structured output using a Pydantic model. It also contains a fallback parsing path for model responses that do not arrive in the expected structure. This allows the agent to recover from formatting differences while retaining the requirement that the resulting RiskItem objects satisfy the defined schema.

10. Project Health Scoring

10.1 Purpose

Project health provides a compact numerical representation of the current state of a project based on the structured information already extracted. The implementation deliberately computes the health score deterministically rather than asking the language model to invent a numerical score. This makes the score reproducible for the same inputs.

10.2 Health Components

Component	Meaning	Weight
Scope Clarity	How completely the available scope information is defined.	30%
Documentation Completeness	How completely the expected documentation is represented.	20%
Risk Density	The proportion of identified risks that are high or critical.	30%
Schedule Risk	A schedule-risk measure based on high/critical schedule risks.	20%

10.3 Formula

The implemented weighted score is:

Health Score = (Scope Clarity × 0.30) + (Documentation Completeness × 0.20) + (Risk Density × 0.30) + (Schedule Risk × 0.20)

The resulting value is rounded to one decimal place and constrained conceptually to the 0–100 range by the report model.

10.4 Schedule Risk Calculation

The implementation identifies risks whose category is schedule. It counts schedule risks with high or critical severity and calculates the schedule-risk component from their proportion among the total risk list. When there are no risks, the schedule-risk component is set to 100. This component is then included in the weighted health score.

10.5 Health Breakdown

The analysis report stores the four component percentages alongside the overall score. The frontend uses these values when the report is available and displays health status categories based on the score: scores of 70 or above are presented as Healthy, scores from 40 to below 70 as Moderate Risk, and scores below 40 as Critical Risk.

11. Automated Project Document Generation

11.1 Document Generation Strategy

The document generation agent converts project evidence and structured analysis into reusable project artefacts. It also supports document auditing so that the system can determine which artefacts are already present and generate only the missing types.

Generated Document	Implementation
Executive Summary	LLM-generated Markdown document based on extracted scope and retrieved project context; structured into project overview, objectives, deliverables, timeline, stakeholders and conclusion.
User Stories	LLM-generated Markdown user stories with title, As-a/I-want/so-that structure, acceptance criteria, priority and estimate; the prompt requests 8–15 stories grouped by feature area.
Risk Register	Generated directly from structured RiskItem objects and formatted as a Markdown table followed by detailed risk descriptions and source context.
Sprint Plan	LLM-generated Markdown sprint plan using project deliverables, timeline and retrieved task/milestone context; the prompt requests 3–5 sprints.

11.2 Document Audit

The document audit checks the AnalysisReport for generated document types already stored in MongoDB. It compares those types with the supported document set and creates a missing-document list. The document-generation node then generates only the missing types. If every supported type already exists, the skip-generation node avoids unnecessary generation.

11.3 Generated Document Retrieval

Generated documents are persisted as part of the analysis report. They are also synchronised into Qdrant in chunked form, allowing the conversational assistant to retrieve generated project information alongside original source documents and the analysis report summary.

12. Conversational Project Intelligence Assistant

12.1 Purpose

The conversational assistant provides natural-language access to project intelligence. It is designed to answer questions about project status, risks, scope, deliverables, generated reports and team progress using the selected project's indexed information.

12.2 Grounding Workflow

24. Receive the user's project-specific question.
25. Select the Qdrant collection belonging to the project.
26. Retrieve the twelve most relevant semantic passages.
27. Generate human-readable source labels from passage metadata.
28. Build a context block containing each source label and passage.
29. Append the most recent six conversation messages when available.
30. Send the system instructions, recent conversation context and retrieved project sources to the language model.
31. Return the response together with deduplicated source labels.

12.3 Grounding Behaviour

The assistant's system instructions explicitly require it to ground responses in the supplied project context and to state when the context does not contain enough information. It is also instructed to highlight relevant risks and blockers and to provide actionable recommendations grounded in available sources.

12.4 Source Labelling

Source labels are created from metadata. A document title is preferred, followed by filename, document type or artifact type. Generated documents are labelled as generated, while the analysis report summary receives a dedicated analysis-report label. This allows the conversational interface to distinguish original project material from derived project artefacts.

13. Project Workspace and Dashboard

13.1 Frontend Project Structure

The current frontend is organised around project-level pages and tabs. The project list allows navigation between workspaces, the project creation page provides project setup, and the project detail page exposes the analytical workspace.

Tab / Area	Purpose
Overview	Project summary, current status, health information, document and analysis actions.
Documents	Upload and manage project documents and access document viewing.
Analysis	Start and monitor the AI analysis pipeline and inspect analytical results.
Chat	Interact with the grounded project assistant.
Generated Data	Inspect generated project documents.
Report	View the stored analysis report.
Project Settings	Manage project-level settings exposed by the frontend.

13.2 Project Status

The frontend recognises project lifecycle states including project created, uploading documents, processing documents, analysis pending, AI analysis running, analysis complete, all pipeline steps complete, and pipeline error. These states allow users to understand whether a project is ready for analysis or still being processed.

13.3 Overview Health Presentation

The Overview tab retrieves the report and uses the stored health breakdown when available. The interface displays the overall health status and provides direct actions to upload documents or run the AI pipeline. This creates a compact entry point into the more detailed analysis and report views.

13.4 Document Viewing and PDF Output

The frontend contains a DocumentViewer component and a DownloadPDF component. Markdown formatting is handled through the frontend's Markdown formatter, and html2pdf.js is included for PDF generation/download support. These components provide a presentation layer for project artefacts rather than changing the underlying analytical data.

14. Data Model and Persistence

14.1 Core Analysis Models

Model	Stored Information
RiskItem	Title, description, category, severity, probability, impact, mitigation and optional source context.
ScopeOutput	Project name, objectives, deliverables, timeline, stakeholders, out-of-scope items and summary.
HealthBreakdown	Schedule risk, scope clarity, documentation completeness and risk density percentages.
GeneratedDocument	Title, document type, Markdown content and creation timestamp.
AnalysisReport	Project ID, analysis status, error message, agent outputs, health score, generated documents, document audit state, pipeline step, raw outputs and timestamps.

14.2 Risk and Scope Schema

The structured schemas are important because they provide a stable contract between the language model, the orchestration layer, persistence, and the frontend. RiskCategory is restricted to technical, resource, schedule, scope, external and quality. RiskSeverity is restricted to low, medium, high and critical. This prevents arbitrary category and severity strings from propagating through the application.

14.3 Analysis Lifecycle

The AnalysisReport model records pending, running, ready and failed states. It also records the current pipeline step, created timestamp and completion timestamp. This supports status polling and makes the state of an analysis run persistent.

14.4 Vector Data

Qdrant stores source-document chunks and derived analytical content. Source chunks contain project and document metadata. The analysis report summary contains project, status, health and scope/risk information. Generated document chunks contain project, source-kind, document type, chunk index, total chunk count and creation time.

15. API and Backend Implementation

15.1 API Responsibilities

The FastAPI application provides the controlled interface between the frontend and backend services. Routes are separated by responsibility, while controllers and services encapsulate application logic. Authentication dependencies can be applied to protected routers so that analytical operations are not directly exposed without user validation.

15.2 Analysis Endpoints

Method	Endpoint Pattern	Purpose
POST	/analysis/{project_id}/run	Start the complete analysis workflow.
POST	/analysis/{project_id}/generate-missing	Generate only missing document types for an existing analysis.
GET	/analysis/{project_id}/status	Return analysis progress and summary information.
GET	/analysis/{project_id}/doc-audit	Audit generated document types.
GET	/analysis/{project_id}/report	Return the stored analysis report.
GET	/analysis/{project_id}/documents	Return generated documents.

15.3 Conversation Endpoint

Method	Endpoint Pattern	Purpose
POST	/chat/{project_id}/message	Submit a project-specific question and return the grounded answer with source information.
GET	/chat/{project_id}/history	Retrieve the stored project conversation where supported by the application contract.
DELETE	/chat/{project_id}/history	Clear the stored conversation where supported by the application contract.

15.4 Long-Running Analysis

The analysis route starts the main workflow as a background task rather than waiting for every AI operation to finish. The frontend can subsequently query the status endpoint. This pattern avoids keeping the initial HTTP request open for the complete document retrieval, model inference, document generation, persistence, and vector synchronisation sequence.

15.5 Backend Modularisation

The backend separates agents, RAG components, routes, models, schemas, storage, configuration, dependencies, controllers and utilities. The structure allows the same project workspace to support conventional CRUD operations and AI-intensive workflows without placing all functionality in a single module.

16. Authentication and Security

16.1 Authentication

The current application includes JWT-based authentication configuration and a dedicated authentication dependency. The authentication layer accepts an access token through the supported cookie or Authorization-header path, validates the token using the configured algorithm and resolves the associated user.

16.2 Configuration

Setting	Purpose
JWT_SECRET_KEY	Secret used to sign and validate access tokens.
JWT_ALGORITHM	Algorithm used for token validation.
ACCESS_TOKEN_EXPIRE_MINUTES	Configured lifetime of access tokens.
MAX_UPLOAD_SIZE_MB	Configured maximum upload size.
API_ORIGIN	Allowed frontend/application origin configuration.

16.3 Data Protection Considerations

- AI service API keys should remain outside source control.
- JWT secrets must be strong and environment-specific.
- Project documents should be treated as potentially confidential information.
- Vector collections and database records should remain inaccessible to unauthorised users.
- Object storage credentials must be protected and rotated appropriately.
- Generated documents should inherit the confidentiality requirements of their source documents.

16.4 AI Security Considerations

Project documents are untrusted inputs from the perspective of a language model. A document can contain text that resembles instructions, so retrieval and prompting should treat document content as evidence rather than executable instructions. The current chat prompt also instructs the assistant to remain grounded and to admit when the available context is insufficient.

17. End-to-End System Workflow

17.1 Complete Workflow

32. User authentication establishes access to the application.
33. User creates or selects a project workspace.
34. User uploads project documents.
35. Document metadata and storage information are recorded.
36. An ingestion task downloads the stored file to a temporary path.
37. Docling converts the document into a common Markdown-oriented representation.
38. Recursive character chunking creates overlapping passages.
39. Metadata is attached to each passage.
40. Embeddings are generated for the passages.
41. Passages are inserted into the Qdrant collection belonging to the project.
42. Document processing status is updated to completed or failed.
43. User starts the AI analysis pipeline.
44. Scope retrieval gathers evidence for objectives, deliverables, timeline, stakeholders and exclusions.
45. Scope agent returns structured project scope.
46. Risk retrieval gathers evidence for project challenges, requirements, schedule, resources, dependencies and technical complexity.
47. Risk agent returns structured risks with severity, probability, impact, mitigation and source context.
48. Health agent deterministically computes the health score and breakdown.
49. Document audit determines which generated document types are already present.
50. Only missing document types are generated.

51. Analysis results and generated artefacts are persisted.
52. Analysis summary and generated documents are synchronised to the project's vector collection.
53. Frontend displays project status, analysis, health, report and generated data.
54. User can ask questions through the project assistant.
55. Chat retrieval searches the same project collection and supplies relevant evidence to the language model.

17.2 Pipeline State

The analytical workflow uses a shared pipeline state. The state carries the project identifier, extracted scope, risk list, health score, health breakdown, generated documents, document audit information, pipeline step information, raw outputs and execution logs. Individual LangGraph nodes return the fields they modify, allowing the orchestration layer to merge the results into the shared state.

17.3 Conditional Generation

The document-generation stage is conditional. The audit node first determines existing and missing document types. If missing types exist, the generation node produces only those types. If no types are missing, the skip-generation path avoids another round of document-generation calls. This design is useful both for efficiency and for incremental project documentation.

18. Testing and Evaluation Strategy

18.1 Testing Approach

Because the platform combines deterministic software with probabilistic AI components, evaluation should be divided into software correctness, retrieval quality, structured-output correctness, generated-content quality, and end-to-end workflow behaviour. A successful HTTP response alone does not establish that the generated project intelligence is correct.

Testing Area	Validation Focus
Authentication	Valid, invalid and expired token behaviour; protected route access.
Project Management	Project creation, retrieval, status and settings.
Document Upload	Supported file handling, metadata persistence and upload-size validation.
Ingestion	Parsing, chunking, metadata enrichment, embedding and Qdrant insertion.
Failure Handling	Failed documents, error recording and temporary-file cleanup.
Scope Extraction	Correctness and completeness of structured scope fields.
Risk Detection	Risk relevance, category, severity, probability, mitigation and evidence.
Health Scoring	Exact formula, boundary cases and reproducibility.
Document Generation	Required sections, consistency with structured inputs and document-type detection.
Chat	Retrieval relevance, grounding, source labels and insufficient-context behaviour.
Frontend	Project navigation, tabs, uploads, status display, report rendering and PDF output.
Deployment	Environment variables, service connectivity and Docker Compose startup.

18.2 Deterministic Tests

- Health-score formula tests using known scope and risk inputs.

- Risk-category and severity schema validation.
- Analysis-status transition tests.
- Document audit tests for all combinations of existing and missing generated documents.
- Chunking tests for overlap and natural separator behaviour.
- Project collection naming and isolation tests.
- Source-label generation tests for uploaded, generated and analysis-report sources.

18.3 AI Evaluation

- Scope extraction completeness.
- Risk precision and relevance.
- Risk evidence attribution quality.
- Retrieval relevance at top-k.
- Groundedness of assistant responses.
- Rate of unsupported claims.
- Generated-document structural completeness.
- Consistency between the stored risk list and generated risk register.

18.4 Historical Evaluation Information

The supplied earlier technical report documented an illustrative three-document execution and recorded the behaviour of the earlier implementation, including multi-query retrieval, risk extraction, deterministic health scoring, document auditing and conditional generation. Those observations are useful as development-history evidence, but they are not treated as current benchmark results unless independently re-run against the current implementation.

19. Results and Discussion

19.1 Functional Result

The implemented application provides a complete project intelligence workflow from project creation and document ingestion to analysis, report persistence, generated documents and conversational retrieval. The current source tree contains dedicated agent implementations for scope, risk, health, document generation and chat, as well as an orchestration layer and pipeline state model. The RAG package contains dedicated document loading, text processing, embedding and ingestion-pipeline components.

19.2 Analytical Result

The analytical output is structured rather than being stored as a single unstructured response. Scope information is represented through defined fields, risks are represented through RiskItem objects, health is represented through a numerical score and breakdown, and generated documents are stored with explicit document types. This structured approach allows the frontend to render individual aspects of the analysis and allows downstream components to retrieve derived information.

19.3 Efficiency Result

The architecture reduces unnecessary context transmission by retrieving only relevant passages for each agent. Scope and risk agents use task-specific retrieval queries rather than one generic search. The chat agent uses a single semantic search over the project collection. Generated documents are chunked and indexed so that later questions can retrieve relevant portions without requiring the entire document to be loaded into context.

19.4 Reusability Result

The project report, generated documents and source documents are all represented within the same project intelligence layer. This means a generated user story or risk register can become part of the retrieval context for later conversational questions. The system therefore behaves as a persistent project knowledge layer rather than a one-time analysis script.

19.5 User Experience Result

The frontend organises project intelligence into separate views rather than presenting one large response. Overview provides status and health information, Documents manages source material, Analysis controls the pipeline, Generated Data displays generated artefacts, Report displays the analytical report, and Chat provides natural-language interaction. This separation matches the underlying data model and supports progressive exploration.

20. Limitations and Future Scope

20.1 Current Limitations

The system's analytical quality depends strongly on the information present in the uploaded project documents. If the documents are incomplete, outdated or contradictory, the resulting scope and risk analysis may also be incomplete. Retrieval-Augmented Generation reduces unsupported generation but does not guarantee perfect interpretation.

The current implementation is also dependent on external AI and vector services. Model availability, embedding service availability, network conditions, and configuration errors can affect analysis. In addition, generated project artefacts remain AI-generated content and should be reviewed before being used as authoritative project records.

20.2 Areas for Future Enhancement

- Introduce richer temporal delivery forecasting using historical task and milestone data.
- Add a dedicated blocker/action-item extraction workflow if required by the project scope.
- Integrate external project management systems such as issue trackers and task platforms.
- Add hybrid retrieval combining semantic similarity with keyword and metadata filtering.
- Add reranking to improve retrieval relevance for complex project queries.
- Introduce a dedicated asynchronous job queue for larger workloads and retry management.
- Add comprehensive automated unit, integration and end-to-end test suites.
- Add detailed observability including structured logs, metrics and tracing for each pipeline node.
- Add stronger role-based access control for multi-user and enterprise scenarios.
- Add versioning of analysis runs and model configurations for auditability.
- Improve document deletion and re-indexing consistency across structured, object and vector storage.
- Provide richer historical health trends and comparative project analytics.
- Expand table and spreadsheet-aware retrieval and analysis.

20.3 Research and Engineering Opportunities

A future version could evolve from document-grounded risk identification toward genuine risk forecasting by combining project-document evidence with historical project outcomes, task completion trends, milestone slippage, dependency graphs and time-series signals. Such a forecasting component would require a suitable historical dataset and a separately validated predictive methodology rather than relying solely on a language model.

21. Conclusion

The AI Project Intelligence & Risk Advisor demonstrates an integrated approach to converting scattered project documentation into structured, searchable and actionable project intelligence. The system addresses a practical problem in software development: important project information exists in many documents, but teams often lack a unified mechanism for interpreting it quickly.

The current implementation combines a React/Vite interface, FastAPI backend, MongoDB persistence, S3/MinIO-compatible document storage, Qdrant vector search, document processing through Docling, semantic embeddings, specialised AI agents, deterministic health scoring, automated project-document generation, and a grounded conversational assistant. These components are connected through a modular application architecture rather than a single monolithic AI prompt.

The RAG layer provides the foundation for project-specific knowledge retrieval. The scope and risk agents use targeted queries and structured output models. The health agent produces a deterministic score that can be reproduced from the same inputs. The document generator turns project evidence into reusable artefacts, while the chat agent exposes the combined project knowledge through natural language.

The resulting platform provides a useful foundation for proactive project awareness. It does not eliminate the need for human project management, but it reduces the manual effort required to discover information, identify documented risks, understand project health, and prepare recurring project artefacts. Future improvements in forecasting, evaluation, observability, integrations and security can extend the platform toward broader enterprise project intelligence use cases.

References

1. Lewis, P., Perez, E., Piktus, A., et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems, 2020.
2. FastAPI Documentation. FastAPI framework documentation.
3. MongoDB Documentation. MongoDB database documentation.
4. Beanie Documentation. Asynchronous Python ODM documentation for MongoDB.
5. Qdrant Documentation. Vector database and semantic search documentation.
6. LangChain Documentation. Components for document processing, embeddings, retrieval and language-model integration.
7. LangGraph Documentation. Graph-based orchestration for stateful AI workflows.
8. Docling Documentation and Project Repository. Document conversion and structured document processing.
9. React Documentation. React frontend development documentation.
10. Vite Documentation. Frontend development and build tooling documentation.
11. Pydantic Documentation. Data validation and typed Python models.
12. PyJWT Documentation. JSON Web Token implementation and validation documentation.
13. NVIDIA AI Endpoints / Embeddings Documentation. Embedding model integration used by the current implementation.
14. Google Generative AI / Gemini Documentation. Language-model integration used by the analytical and conversational agents.

Appendix A – API Summary

Area	Endpoint / Operation	Purpose
Project	Project creation/list/detail operations	Manage project workspaces.
Documents	Document upload/list/status/view/delete operations	Manage source project documents.
Analysis	POST /analysis/{project_id}/run	Start complete analysis.
Analysis	POST /analysis/{project_id}/generate-missing	Generate missing document types.
Analysis	GET /analysis/{project_id}/status	Monitor processing.
Analysis	GET /analysis/{project_id}/doc-audit	Inspect generated-document completeness.
Analysis	GET /analysis/{project_id}/report	Retrieve full analysis.
Analysis	GET /analysis/{project_id}/documents	Retrieve generated documents.
Chat	POST /chat/{project_id}/message	Ask a grounded project question.
Chat	GET /chat/{project_id}/history	Retrieve conversation history.
Chat	DELETE /chat/{project_id}/history	Clear conversation history.

Appendix B – Configuration and Deployment

B.1 Backend Configuration

Variable / Area	Purpose
MONGODB_URI	MongoDB connection address.
DB_NAME	Application database name.
QDRANT_URL	Qdrant vector database address.
GOOGLE_API_KEY	Credential for configured Google generative AI services.
NVIDIA_API_KEY	Credential for the current NVIDIA embedding implementation.
LLM_MODEL	Configured language model identifier.
HF_TOKEN	Document-processing/model-stack credential where required by the configured environment.
BACKEND_HOST / BACKEND_PORT	Backend binding configuration.
API_ORIGIN	Frontend origin allowed by the backend configuration.
MAX_UPLOAD_SIZE_MB	Maximum upload size configuration.
JWT_SECRET_KEY	JWT signing/validation secret.
JWT_ALGORITHM	JWT algorithm.
ACCESS_TOKEN_EXPIRE_MINUTES	Access-token lifetime.
S3_ENDPOINT	S3/MinIO-compatible storage endpoint.
S3_ACCESS_KEY / S3_SECRET_KEY	Object-storage credentials.
S3_BUCKET	Bucket used for documents.

B.2 Development Sequence

56. Install Python and Node.js prerequisites.
57. Configure MongoDB and Qdrant.
58. Configure the backend environment variables.
59. Install backend dependencies.
60. Start the FastAPI backend.
61. Configure the frontend API endpoint.
62. Install frontend dependencies.
63. Start the Vite development server.
64. Open the application and authenticate.
65. Create a project, upload documents, run analysis and inspect the results.

B.3 Container Deployment

The project includes backend and frontend Dockerfiles and a Docker Compose configuration. The documented container workflow starts the supporting services together. Before using the composition beyond controlled evaluation, deployment-specific requirements such as secret management, persistent object storage, network restrictions, monitoring, backups, and production process management should be validated.

Appendix C – Data Structures and Output Formats

C.1 Risk Object

A risk entry contains a title, description, category, severity, probability, impact, mitigation, and optional source context. The category and severity fields are constrained enumerations.

C.2 Scope Object

A scope object contains project name, objectives, deliverables, timeline, stakeholders, out-of-scope information, and a concise summary.

C.3 Health Breakdown

The health breakdown stores schedule-risk percentage, scope-clarity percentage, documentation-completeness percentage, and risk-density percentage.

C.4 Generated Document

A generated document contains a title, document type, Markdown content, and creation timestamp. Supported document types in the current model are `executive_summary`, `user_stories`, `risk_register`, and `sprint_plan`.

C.5 Analysis Report

The analysis report combines project identifier, status, error information, scope, risks, health score, health breakdown, generated documents, existing and missing document types, pipeline step, raw outputs, and timestamps.

Appendix D – Glossary

Term	Meaning
AI	Artificial Intelligence; computational methods used for analysis and generation.
Agent	A specialised processing component responsible for a defined analytical task.
Embedding	A numerical vector representation of content used for semantic similarity.
RAG	Retrieval-Augmented Generation; retrieval of relevant evidence before language-model generation.
Vector Store	Storage system optimised for vector similarity search.
Qdrant	Vector database used for project-specific semantic collections.
Chunk	A bounded section of processed document text used as a retrieval unit.
Grounding	Constraining generated responses to retrieved project evidence.
Scope	Structured description of project objectives, deliverables, timeline, stakeholders and exclusions.
Risk Item	Structured representation of an identified project risk.
Health Score	Deterministic numerical representation of project health based on defined components.

Generated Document	AI-generated or programmatically formatted project artefact stored in the analysis report.
Pipeline State	Shared data structure passed between analysis workflow nodes.
Semantic Search	Retrieval based on vector similarity rather than exact keyword matching.
JWT	JSON Web Token used for token-based authentication.
ODM	Object-Document Mapper used to represent database documents as application models.
MongoDB	Document-oriented database used for structured application state.
S3/MinIO	Object-storage interface used for project document storage.

Appendix E – Implementation Verification Notes

This report has been written to distinguish the original project statement from the functionality that is represented in the current implementation. In particular, the current codebase clearly contains scope extraction, risk analysis, deterministic health scoring, document generation, RAG-based conversational assistance, document ingestion, vector indexing, project management, authentication, report persistence, and the corresponding frontend views. A separate standalone blocker/action-item agent is not represented in the current agent package, and therefore it is not presented in this report as a completed module.

Similarly, the current implementation contains schedule-oriented risk analysis and a schedule-risk component within the health calculation. This report does not describe a separate trained time-series forecasting model because such a component is not represented in the reviewed implementation. The project statement's broader objective of proactive risk awareness is therefore documented through the completed risk-analysis and health-scoring capabilities.

The report should be updated if the implementation changes before final submission. In particular, exact model identifiers, deployment configuration, measured performance results, screenshots, and test results should be refreshed from the final running version rather than copied from historical documentation.